Table for Four — Project Scoping & Initial Agent Design

CMU AI Agent Certification — Capstone
Author: Manish Bhatt
Document version: 1.1 · updated through the web-highlights layer
Status: Milestones 1–3.6 complete — search, booking backend, perks/RAG, LangGraph orchestrator, conversational concierge with long-term memory, and live web highlights. Next: the human-approval gate and governance/audit layer (M4).

---

1. Executive summary

Table for Four is an agentic restaurant concierge. A user makes a natural-language dining request — *"Italian, near Midtown, 4 people, Friday 7pm, one guest is gluten-free"* — and the agent searches real restaurant data, reasons over which options fit, surfaces relevant perks (coupons/offers), and books a table through a reservation service. A **human-in-the-loop confirmation gate** sits before any booking is finalized, and a **governance/audit layer** records every tool call and approval.

The system is deliberately built **MCP-first** (tools exposed as Model Context Protocol servers) and **orchestrated with LangGraph**, so the reasoning loop, tool use, human approval, and auditing are explicit and inspectable rather than hidden inside a single prompt.

> **Built with an agent, about an agent.** The project is itself scaffolded and developed using an agentic coding tool (Claude Code in VS Code) — a live, in-workflow demonstration of the same technology the capstone studies.

---

2. Problem statement

Booking a restaurant for a group is a small but real multi-constraint decision problem. A person must reconcile cuisine, location, party size, timing, budget, dietary restrictions, ratings, and availability — then act on it (book) — often across several tabs and apps. It is:

- **Multi-constraint:** several soft and hard constraints interact.
- **Retrieval-heavy:** the right answer depends on current, real-world data.
- **Transactional:** it ends in an irreversible action (a reservation) that a responsible system should not take without explicit human confirmation.

This makes it an ideal, well-bounded testbed for an agent that must **search, reason, personalize, and act under human oversight** — exercising the full agent loop rather than a single retrieval or generation step.

---

3. Goals & non-goals

3.1 Goals

- Turn a free-text dining request into a **ranked, reasoned set of options** using **real** restaurant data.
- **Personalize** those options with relevant perks/offers via **semantic retrieval (RAG)** over an unstructured perks store.
- **Book** a table through a reservation interface, gated by **explicit human confirmation**.
- Produce a **complete audit trail** of tool calls, data provenance, and approvals.
- Keep the system **runnable offline with no API keys** for development, testing, and grading, while supporting **live data** when keys are provided.

3.2 Non-goals (explicitly out of scope)

- **Real transactional booking** against OpenTable/Resy/SevenRooms. These are partner-gated; we use a **self-built mock reservation backend** by design (see §7).
- Payment processing, real coupon redemption, or any real financial transaction.
- A production web/mobile UI. The interface is a chat/CLI-level interaction sufficient to demonstrate the agent loop.
- Multi-city scale, real-time availability guarantees, or account/identity systems beyond a lightweight member-preferences notion (stretch goal).

---

4. Target user & primary scenario

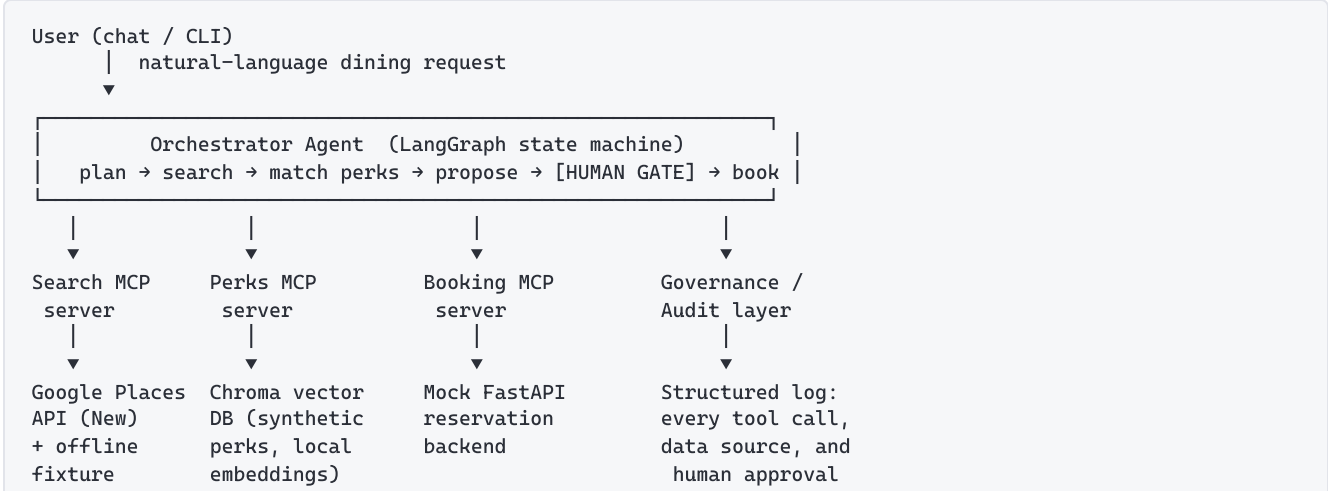
Primary user: an individual arranging a meal for a small group who wants a single natural-language request handled end-to-end.

Primary scenario (happy path):

1. User: *"Italian, near Midtown, 4 people, Friday 7pm, one guest is gluten-free."*
2. Agent searches restaurants, applies constraints (cuisine, area, price, rating, open-now), and returns a shortlist with reasoning.
3. Agent retrieves and attaches **relevant perks** ("2 of these 3 have offers that fit a gluten-free group of 4").
4. User picks an option.
5. Agent proposes a booking and **pauses for explicit confirmation**.
6. On approval, the booking is finalized and a confirmation returned.
7. Every step — searches, perk matches, the approval, the booking — is logged.

---

5. System architecture



Design principles

- **MCP-first.** Every external capability (search, perks, booking) is a discrete MCP tool server. The orchestrator only knows tools, not vendors — swappable and independently testable.
- **Offline-first, live-optional.** Each data path has a keyless offline mode (fixtures / synthetic data / mock backend) and a live mode behind an env var. The **same normalization code runs in both modes**.
- **Provenance is first-class.** Every result carries a `source` field (`live` | `fixture` | `synthetic`) so the audit layer can record what a decision rested on.
- **The irreversible step is gated.** No booking is finalized without an explicit human approval event.

---

6. Agent design detail

6.1 Orchestration (LangGraph)

The agent is modeled as an explicit **state graph**, not a single prompt loop. Nodes as built:

NODE	RESPONSIBILITY
`parse`	Extract structured constraints from free text (cuisine, area, party size, time, dietary, budget).
`search`	Call the Search MCP tool; get candidate restaurants.
`refine`	On an empty search, relax **one** constraint and loop back to `search`. Bounded by `MAX_ITERATIONS` so it can never spin.
`match_perks`	Call the Perks MCP tool with the user's intent + candidate `place_id`s; attach fitting offers.
`rank`	Reason over fit; produce a ranked shortlist with rationale.
`propose`	Draft a booking for the chosen option.
`gate`	**Interrupt**; present the booking for explicit approval/decline. *(Auto-approves today; the real interrupt is M4.)*
`book`	On approval, call the Booking MCP tool; return confirmation.
`audit`	Log the run — reached whether or not the booking went ahead.

State is carried explicitly between nodes so the flow is inspectable and resumable — a core reason for choosing LangGraph over an implicit agent loop.

6.2 Tools (MCP servers)

A. Search — `search_restaurants` *(built, Milestone 1)*

- Backed by **Google Places API (New)**, with an **offline fixture** fallback.
- Normalizes raw results to a clean shape (`place_id`, `name`, `address`, `price_level` 0–4, `rating`, `open_now`, `location`, …).
- Server-side-style filters: `max_price_level`, `min_rating`, `open_now`, `max_results`.
- Uses an explicit **field mask** so live calls fetch only needed fields — this controls both the response shape and the **billing SKU** (cost control).
- Returns `source: "live" | "fixture"`.

B. Perks — `find_perks` *(built, Milestone 2.5)*

- Backed by a **Chroma vector database** of **synthetic** perks.
- **Hybrid retrieval:** semantic vector similarity over an unstructured `blurb` (cuisine/vibe/dietary/occasion) **combined with** structured metadata filtering (`min_party_size`, `expiry`, `dine_in_only`, `perk_type`, `discount_pct`, `valid_days`, `active`).
- **Local embeddings** (`all-MiniLM-L6-v2`) — no API key, fully offline.
- Perks are keyed to restaurants by `place_id`. Returns match score and `source: "synthetic"`.

C. Booking — `create_booking` / `check_availability` / `cancel_booking` *(built, Milestone 2)*

- Backed by a **self-built mock FastAPI reservation backend** (see §7).
- Exposes availability lookup and reservation creation with a confirmation id.

- Deterministic/mocked so the transactional path is fully testable offline.
- Cancellation enforces the **24-hour policy in the backend**, not the model: inside the window it refuses and returns the restaurant's phone/website instead.

D. Web highlights — `lookup_dining_highlights` *(built, see §9.2)*

- Backed by **Tavily** web search, with an offline fixture fallback (placeholder graphics) so the journey still runs with no key.
- Returns cited menu highlights plus photos for a restaurant already recommended or booked — never an open-ended web search (that would breach the dining-only scope).
- Scoped to the restaurant's **own domain first**, widening to the open web only when that returns nothing about the food; photos lifted from the matched pages are preferred over a generic image search, which is what keeps a chain on the right branch.
- Feeds the **generated menu cards** (§9.1), whose dish lines are strictly extractive from retrieved text.

6.3 Human-in-the-loop gate

Before any booking is written, the graph **interrupts** and surfaces the proposed reservation (restaurant, time, party size, any applied perk) for the user to **approve or decline**. Approval is a recorded event; a decline routes back to selection. This makes the one irreversible action a deliberate, auditable human decision — the central responsible-AI control of the system.

6.4 Governance & audit layer

A cross-cutting layer records a structured entry for every consequential step:

- which **tool** was called, with what arguments;
- the **data source/provenance** (`live` / `fixture` / `synthetic`);
- the **human approval** (or decline) event and timestamp;
- the final **booking outcome**.

This yields an end-to-end trail explaining *why* the agent did what it did and *on what data* — directly addressing accountability and transparency requirements.

6.5 Memory model

The agent uses three distinct tiers of memory; naming them separately keeps their lifetimes and responsibilities clear.

TIER	HOLDS	IMPLEMENTATION	MILESTONE
Working memory	The in-flight request: parsed constraints, candidate restaurants, matched perks, the chosen option, the pending booking.	LangGraph **typed** `State` carried across nodes, persisted by a **checkpointer** keyed per conversation **thread**. Enables the human-gate **pause/resume**.	M3 / M4
Long-term profile memory	Member preferences: name/pronouns, email, dietary defaults, favored cuisines, party size, kids, and past bookings.	A **Chroma** store keyed by email, with two retrieval modes (key lookup + semantic recall). Read when a guest is recognised, written as they talk. Cuisines are a **rolling window of the last three** — taste drifts — while dietary needs are never aged out.	✅ Built (M3.5)
Audit / episodic memory	Every consequential event: tool call + args, data `source`, human approval/decline, booking outcome.	Append-only **governance log** (§6.4); doubles as booking history.	M4

6.6 Control flow & reasoning loops

The orchestrator is a **state graph**, not a single prompt — control flow is explicit and inspectable.

- **Happy path (linear):** `parse_request → search → match_perks → rank_and_explain → propose_booking → [human_gate] → book → audit`.
- **Reasoning loops (conditional edges):** the graph can **refine and retry** — e.g. if `search` / `match_perks` return nothing that satisfies the constraints, `rank_and_explain` may relax or rewrite the query and route **back** to `search`. A **max-iteration guard** bounds this so the loop cannot spin. This refine-retry cycle is where the agent's reasoning is exercised, beyond a single linear pass.
- **Human interrupt:** at `human_gate` the graph **interrupts** and waits; on approval it **resumes** from the checkpoint, on decline it routes back to selection. The pause/resume is powered by the working-memory checkpointer.
- **Tools as a registry:** each capability is an independent **MCP server** and the orchestrator is an **MCP client**. Adding a capability means adding a server, not rewriting the graph. Current tools: `search_restaurants` (M1), `find_perks` (M2.5), and `check_availability` / `create_booking` (M2).

7. Data strategy

CAPABILITY	SOURCE	RATIONALE
Restaurant search	**Real** — Google Places API (New), with offline fixture	Real-world retrieval is core to the value; fixture keeps it keyless/testable.
Perks / coupons	**Synthetic**, clearly labeled	No cleanly-licensable free coupon dataset exists; scraping real coupon sites is ToS-gated and weakens the governance story. Labeled synthetic data is the more defensible, reproducible choice.
Booking backend	**Self-built mock** (FastAPI)	Real reservation systems (OpenTable/Resy/SevenRooms) are partner-gated; Yelp dropped its free tier. Mocking a partner-gated downstream is a standard, defensible agent-prototyping pattern.

Provenance honesty. Every payload carries an explicit `source`. The system never presents synthetic or mock data as if it were live — this labeling *is* part of the responsible-AI design, not an afterthought.

Synthetic perks generation. A seeded (reproducible) generator produces several perks per fixture restaurant, spanning perk types, with some deliberately **expired** and some **party-size-gated** (to prove the metadata filters work) and varied dietary/occasion themes (to give semantic retrieval real signal).

8. Technology stack

CONCERN	CHOICE	WHY
Language / runtime	Python ≥ 3.12, managed by **uv**	Fast, reproducible, lockfile-based env.
Tool protocol	**MCP** (`mcp[cli]` , FastMCP)	Standard, inspectable tool interface; decouples agent from vendors.
Orchestration	**LangGraph**	Explicit, resumable state graph with a native interrupt for the human gate.
Vector store	**Chroma** (local, persistent)	Simple local RAG with built-in embeddings; no external service.
Embeddings	`all-MiniLM-L6-v2` (local)	Free, offline, no API key — consistent with offline-first philosophy.
Booking backend	**FastAPI** (mock)	Lightweight, self-contained transactional service.
HTTP	`httpx`	Async-capable modern client for the Places call.
Config	`python-dotenv`	Keeps keys out of code and version control.
Testing	`pytest`	Offline test suites per component.

9. Milestone plan

#	MILESTONE	STATE
1	Search MCP server (Google Places, offline-testable)	✅ Complete
2	Mock booking FastAPI + booking MCP server	✅ Complete
2.5	**Perks / RAG:** synthetic perks → Chroma → `find_perks` MCP tool	✅ Complete
3	LangGraph orchestrator; end-to-end happy path (search → perks → book)	✅ Complete
3.5	Conversational concierge (Dino) + long-term member memory in Chroma	✅ Complete
3.6	Web highlights via Tavily + generated menu cards (§9.1, §9.2)	✅ Complete
4	Human-in-the-loop gate + governance/audit logging	⬜ Next
5	*(Stretch)* model comparison, polished demo	⬜

9.2 Web enrichment via Tavily *(built, ahead of M4)*

Shipped as `lookup_dining_highlights` (§6.2 D) — the tool that adds the *qualitative* context Google Places doesn't carry: signature dishes, what a room looks like, what diners keep mentioning. Built earlier than planned because it is what makes the chat feel like a concierge rather than a booking form.

- **Scope:** enrichment only; it does **not** replace Places for core search (Places is more structured for constraint matching).
- **Bounded to the conversation:** it will only look up a restaurant already recommended or booked in this session, so it cannot become a general web-search back door around the dining-only guardrail.
- **Offline behaviour:** rather than being live-only as first scoped, a fixture path returns seeded highlights with locally generated placeholder graphics, so the whole journey still demos with no key.
- **Representational integrity:** every snippet carries its source domain, photos are captioned with where they came from and never presented as the restaurant's own, and the model is instructed to attribute dishes ("diners keep mentioning…") rather than promise them, since menus change.

9.1 Restaurant imagery *(built, delivered differently than scoped)*

The intent — a richer visual surface that respects representational integrity (§10) — shipped, but not as a photo library. Two layers, both avoiding the original plan's weakness (a folder of stock images that must never be mistaken for a real venue):

- **Generated menu cards** (`agent/menu_card.py`): one cuisine-themed SVG card per restaurant, carrying the retrieved dish highlights and the perk. **Six themes**, rendered as data URIs — no image files, no storage question, works offline. Nothing depicts a specific venue, so nothing can misrepresent one.
- **Real photos, when they exist**, from the web-highlights tool (§9.2), preferring images lifted from the pages actually about that restaurant, each captioned with its source domain.

- **Honesty rule:** every dish line on a card is a **substring of retrieved text**. Where a menu isn't published online, the card says so rather than inventing a plausible dish.
- **Storage:** unchanged from the original scoping — image bytes are **never** put in the vector DB.
- *(Optional stretch-within-stretch:* multimodal CLIP embeddings to enable visually-similar retrieval — the one design that would let images genuinely earn a place in the vector store.)

---

10. Responsible AI & governance considerations

- **Human authority over irreversible actions.** Bookings require explicit confirmation; the agent proposes, the human disposes.
- **Data provenance & honesty.** `source` labeling on every payload; synthetic and mock data are never disguised as live.
- **Auditability.** A structured trail of tool calls, sources, and approvals makes every decision explainable after the fact.
- **Cost control & least-privilege data access.** The Places field mask requests only the fields the agent reasons over, minimizing both data collected and billing.
- **Secret hygiene.** API keys live only in a gitignored `.env`; the recommended key is restricted to the single API it needs, capping blast radius if leaked.
- **Reproducibility.** Offline modes and a seeded synthetic generator make runs deterministic and gradable without credentials.
- **Representational integrity (imagery).** Any AI-generated or stock restaurant image is **illustrative only** and explicitly labeled as such — the system never fabricates a picture and presents it as a depiction of a specific, real, named restaurant. Only *live* Google Places photos are shown as actual images of a place. This guardrail is a first-class design constraint on the Milestone 5 imagery layer, not an afterthought.

---

11. Risks & mitigations

RISK	MITIGATION
Live vs. synthetic **join-key mismatch** (Places `place_id`s differ from fixture ids, so perks won't match live results)	Key perks to the fixture set for the offline demo; decide the live-matching strategy explicitly before wiring M3.
Vector DB used for its own sake (perks that are purely structured don't need embeddings)	Hybrid design: embed genuinely unstructured blurbs, filter on structured metadata — vector search only where semantics add value.
Heavier dependency (`chromadb` pulls in onnxruntime)	Accepted; local embeddings avoid an external service and keep the offline story intact.
Over-automation of the **irreversible booking** step	Non-negotiable human gate before any write; declines are first-class.
Misrepresentation via imagery (a fabricated image implying it depicts a specific real restaurant)	Illustrative-only, clearly-labeled generic imagery offline; real Google Places photos only in live mode; never caption a synthetic image as a specific named venue (see §10).
Scope creep from stretch ideas	Milestones are ordered; stretch items (M5) only after the core loop works end-to-end.

12. Success criteria

The capstone is successful when:

1. A natural-language request flows end-to-end: **parse → search → perk match → proposal → human approval → booking confirmation.**
2. The system runs **fully offline with no API keys** (fixtures + synthetic perks + mock backend) and **switches to live search** when a key is present.
3. **Perk matching demonstrably uses both** semantic similarity and metadata filters (e.g. a gluten-free group-of-4 query surfaces a fitting, non-expired, party-size-valid offer).
4. No booking is ever finalized without a recorded **human approval**.
5. A **governance/audit trail** exists for every consequential step with its data provenance.
6. Each component ships with **passing offline tests**.

End of document — v1.0.